

“Immutability” of systems, and how to break them!

Yvan Sraka

~yvan

Over the past years I worked mainly as a Nix(OS) consultant¹, and gave a lot of talks about it².

You can ask me about the NLNet grant I just got to **speed up nixpkgs evaluation!**

And I use Mastodon, like all the cool kids:

<https://functional.cafe/@yvan>

¹I have a company based in Brussels, Newtype SRL, you can hire me!

²<https://git.sr.ht/~yvan-sraka/nix-talks>

Back in the day, if you said “*I broke my ...*”

- **Phone:** I'd first assume it's **hardware** ...
- **Computer:** I'd first assume it's **software** ...

There were some tradeoff between **reliability** (*achieved by simplicity*) VS **customization** (*if you want to tweak it, you accepted to take the risk of breaking it*)³

Phones and computers have now merged into the same kind of device.⁴

³Seems to be what most Linux users still assume!

⁴I can install apps on my iPhone 2 but I can't tweak the look of window decorations on macOS 11...

Just forbid breaking from everyday operations!

We have **normal** users and **super**users⁵ (*a.k.a. root*).

It's a good model, right? What could go wrong?

⁵Rarely more than one.

Oh no

I clicked on a button that made Ubuntu start an `apt dist-upgrade` but I ran out of battery⁶ before it finished...

⁶We're in a Linux world where too few people set up hibernation.

What if we could make the software never break?

Switching from version A to version B of the system should be **atomic, recoverable, predictable**.

Just have both systems A and B on the same machine!

That's basically how *most* of those “*immutable*” systems work!

And then you can pick the one you want at boot, and have some fallback mechanism, etc.

How do they work?

- 1. Some do real partitioning,
- 2. Some rely on specific filesystem features⁷)
- 3. Some use a “store”⁸

⁷e.g. APFS snapshots, openSUSE MicroOS Btrfs, etc.

⁸That's compatible with filesystems that don't offer Copy-on-Write!

Enforcing “read-only” mode?

We can use the immutable bit? we can sign images? have a trusted boot sequence, etc.

*Wasn't this a talk about **NixOS**? (or Guix?)* Yes!

Input-Addressed Store VS Content-Addressed stores

- A `/nix/store` is **input-addressed**, store object (*a.k.a a derivation*) can only reference other store objects, all the way down! This require some tricks, like rewriting dynamic library paths with `patchelf`,
- **Snix** offers a Nix-compatible frontend but offers a **content-addressed** and so deduplication (*using Merkle-Trees*⁹), and so “trusting” binary caches would be a no brainer.

A Nix derivation (*because its hashes is computed from its inputs*) is better be realized inside a sandbox¹⁰.

⁹What your favorite P2P protocol likely use!

¹⁰Which should give you reproducibility for free, nice :)

No ACLs (yet?) ...

Every user can ask the daemon to **add a new entry** to the store, every user can **read the whole content** of the store, and `/nix/store` (or `/gnu/store`) is owned by root... so:

- you shouldn't write unencrypted secrets in the store,
- you can't set the `setuid` bit to a store path!

But it's still great because ...

- **Specializations!** Using generations¹¹ to have more than one “flavor” of your device!
- We don't even need a trusted binary-cache, actually we even can do a **full-source bootstrap**¹²!
- You can describe your whole systems in a few files, using a surprisingly nice configuration language for ruling all your softwares :)

IMHO it's the best of both worlds between “*flexibility*” and “*reliability*”!

¹¹System “versions” that appear as your bootloader entries.

¹²<https://guix.gnu.org/en/blog/2023/the-full-source-bootstrap-building-from-source-all-the-way-down/>

Q/A

Q/A

Thank you! If you want to visit Toulouse (*South of France*) mid-November, we're organizing a Nix & Guix devroom at **Capitol du Libre**, and the CFP is currently opened :)